

Anonymous Credential in Sphinx Packets

Aurelien Chassagne

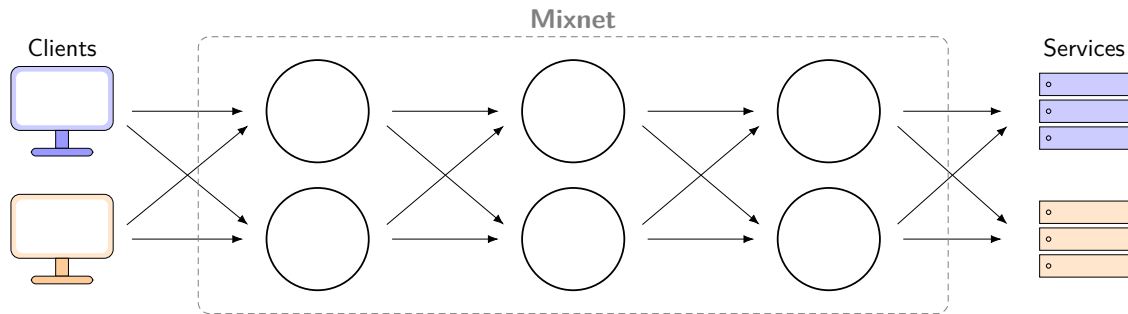
June 16, 2026

Table of Content

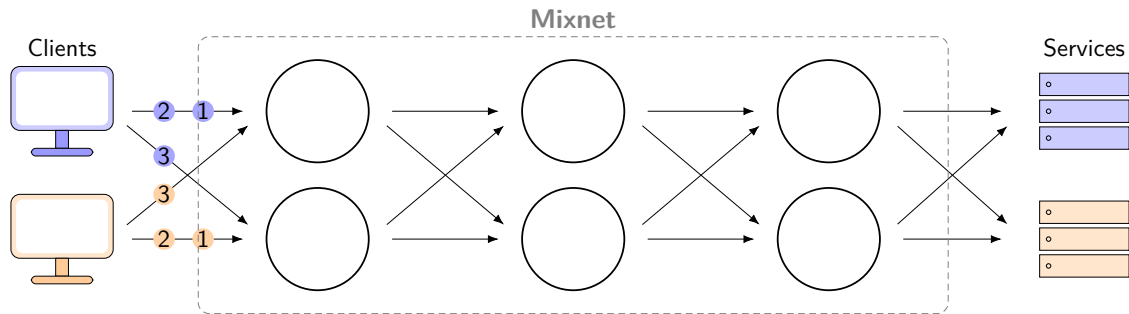
- Background
 - Mixnet - Anonymous Network Communication
 - Sphinx - Packet Format
- My research
 - DSphinx - A more flexible packet format
 - Credential - Verifiable DSphinx Destination
- Implementation & Early Results
- Future work

Background

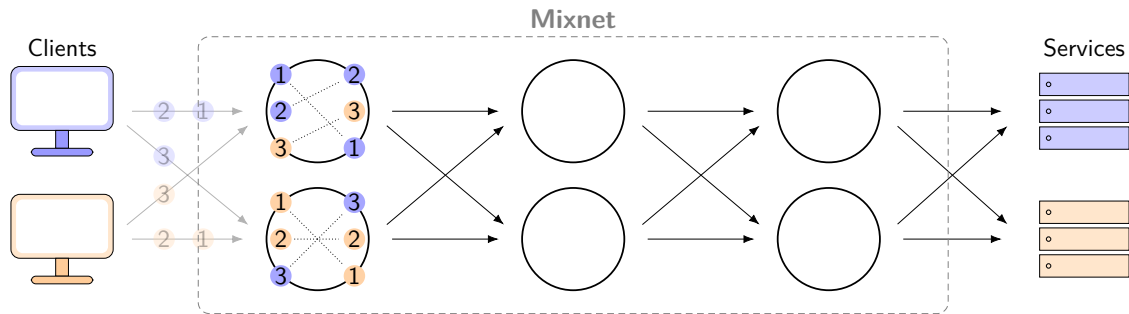
Mixnet - Anonymous Communication



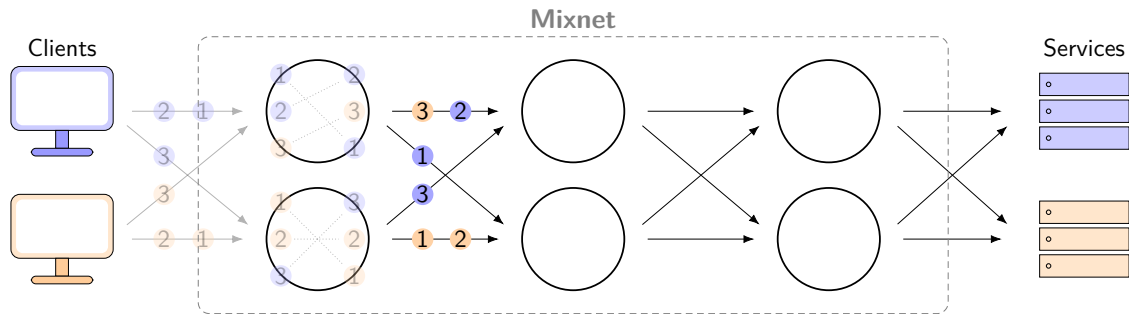
Mixnet - Anonymous Communication



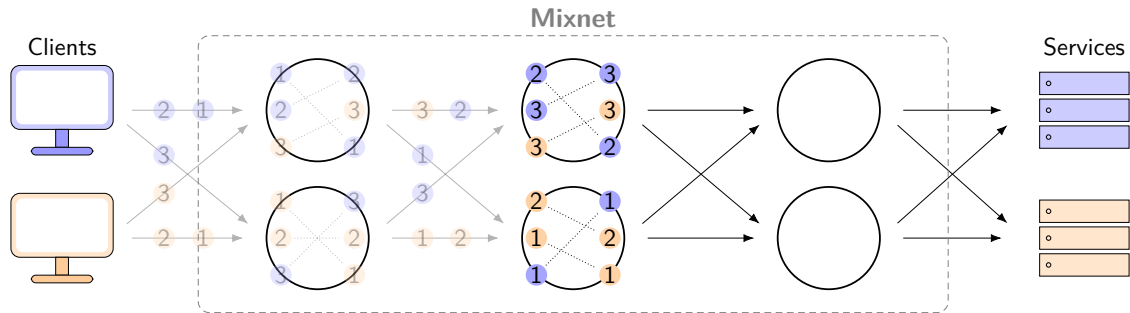
Mixnet - Anonymous Communication



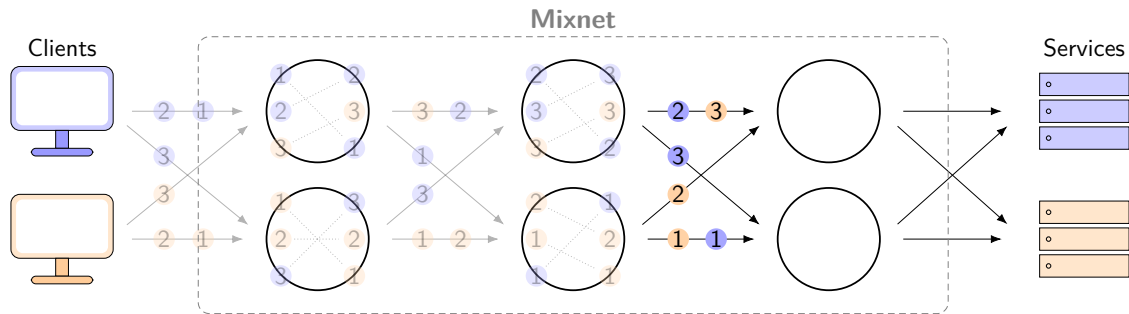
Mixnet - Anonymous Communication



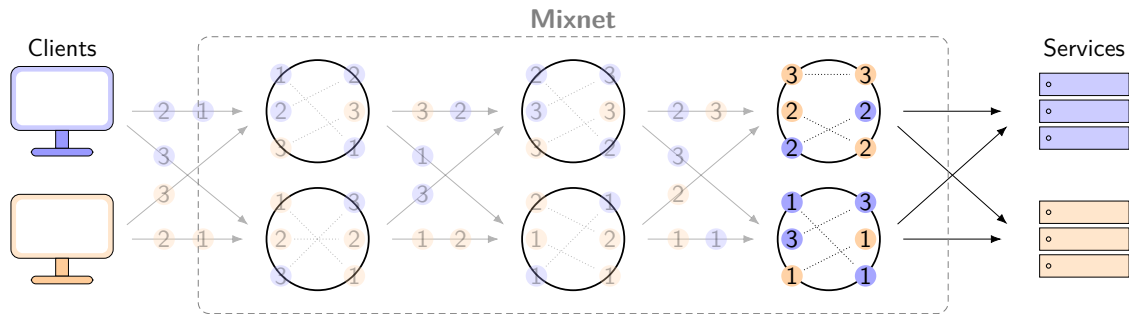
Mixnet - Anonymous Communication



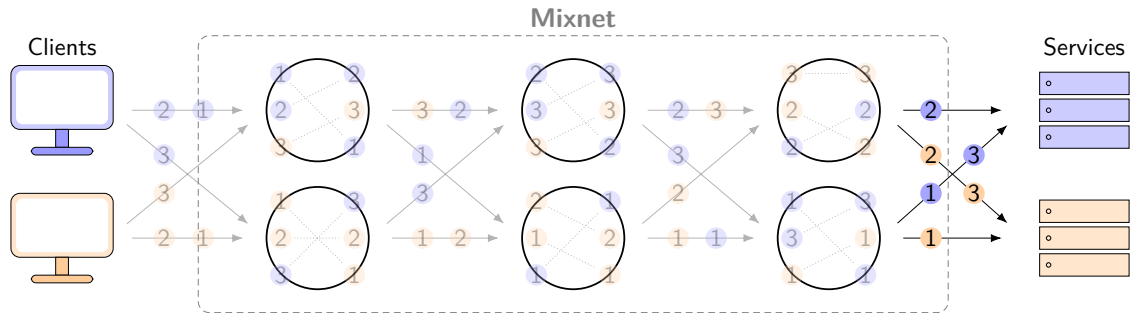
Mixnet - Anonymous Communication



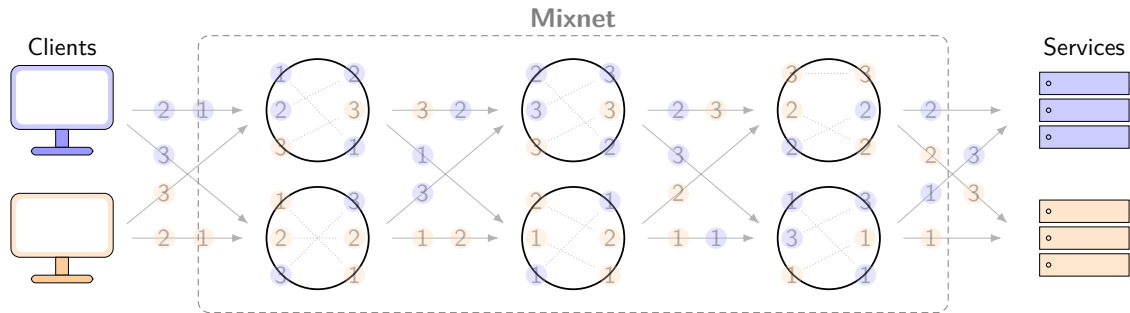
Mixnet - Anonymous Communication



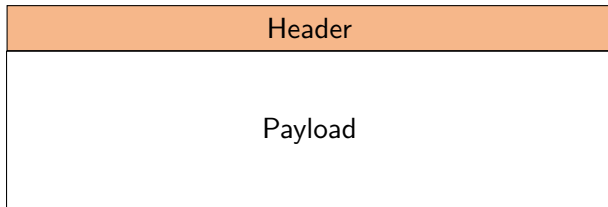
Mixnet - Anonymous Communication



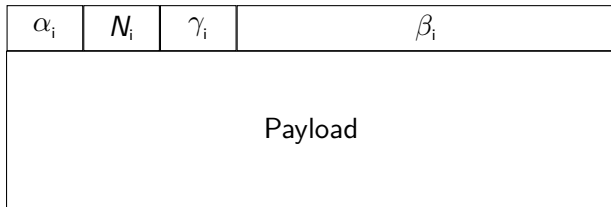
Mixnet - Anonymous Communication



Sphinx - Packet format



Sphinx - Packet format



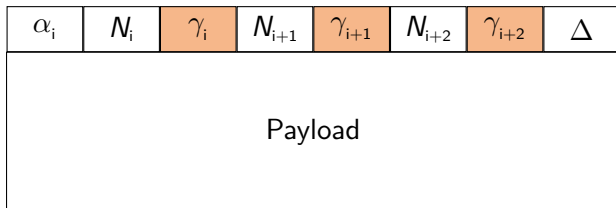
α : Cryptographic element

N : Next Hop

γ : Integrity tag

β : Routing information

Sphinx - Packet format



Properties:

α : Cryptographic element

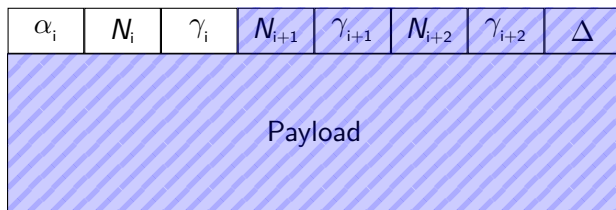
N : Hops in the path

γ : Integrity tag

β : Routing information

- Per-hop Integrity

Sphinx - Packet format



Encryption



Layer 1



Layer 2



Layer 3

Properties:

α : Cryptographic element

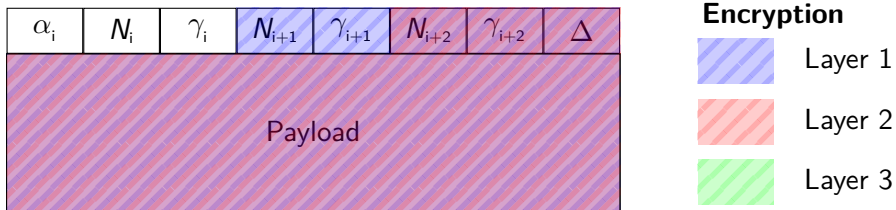
N : Hops in the path

γ : Integrity tag

β : Routing information

- Per-hop Integrity
- Per-hop confidentiality

Sphinx - Packet format



Properties:

α : Cryptographic element

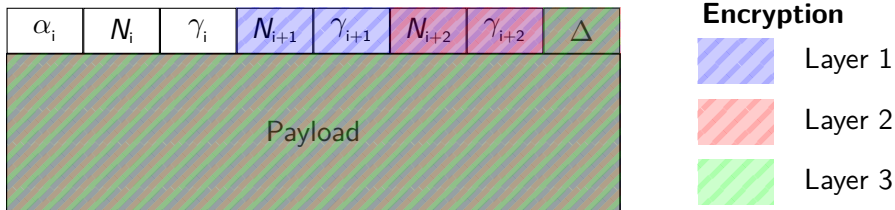
N : Hops in the path

γ : Integrity tag

β : Routing information

- Per-hop Integrity
- Per-hop confidentiality

Sphinx - Packet format



Properties:

α : Cryptographic element

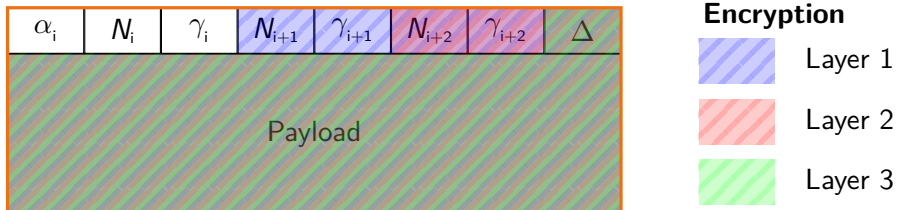
N : Hops in the path

γ : Integrity tag

β : Routing information

- Per-hop Integrity
- Per-hop confidentiality

Sphinx - Packet format



α : Cryptographic element

N : Hops in the path

γ : Integrity tag

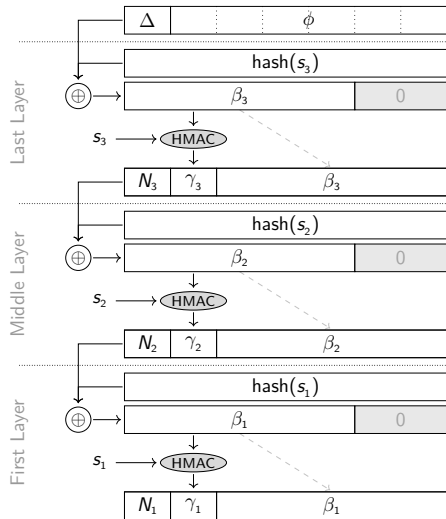
β : Routing information

Properties:

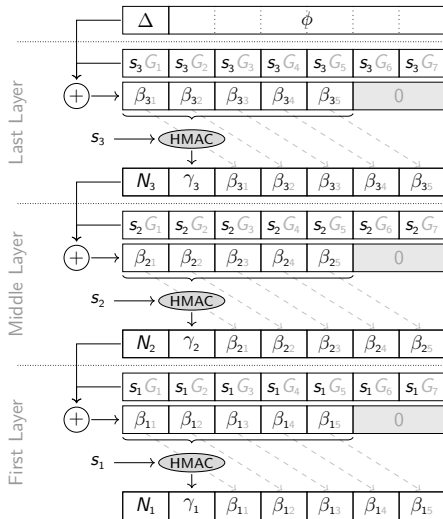
- Per-hop Integrity
- Per-hop confidentiality
- **Strong unlinkability**
- **Fixed-size packet**

My Research - DSphinx

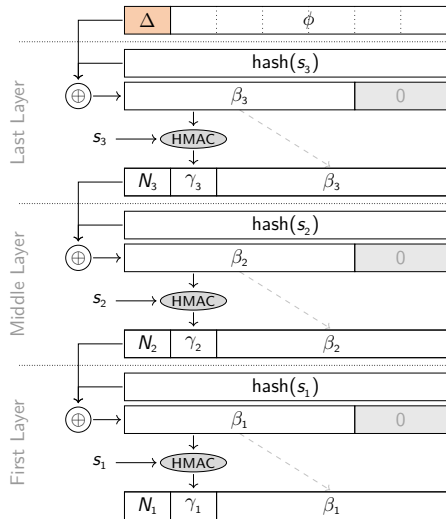
From Sphinx



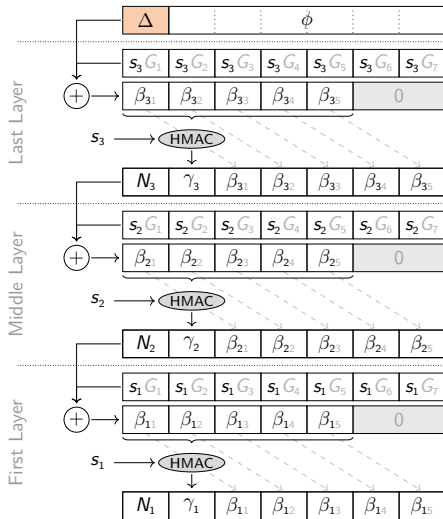
To DSphinx



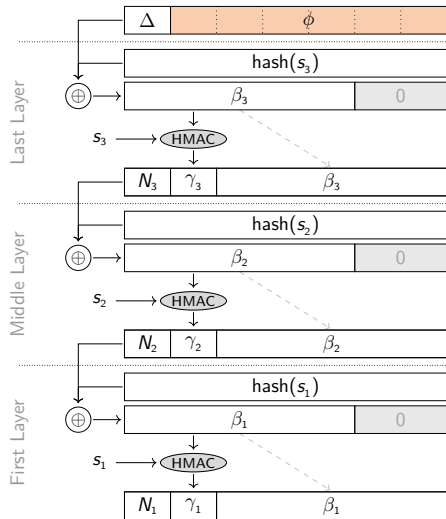
From Sphinx



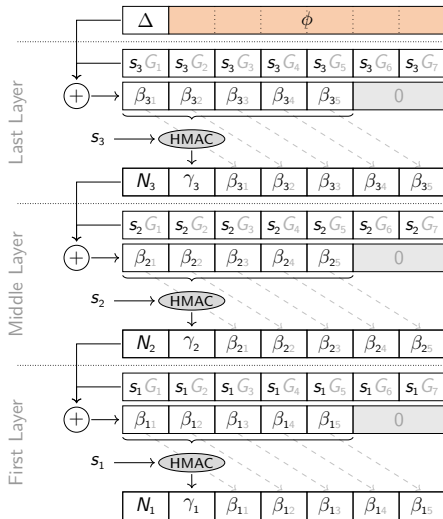
To DSphinx



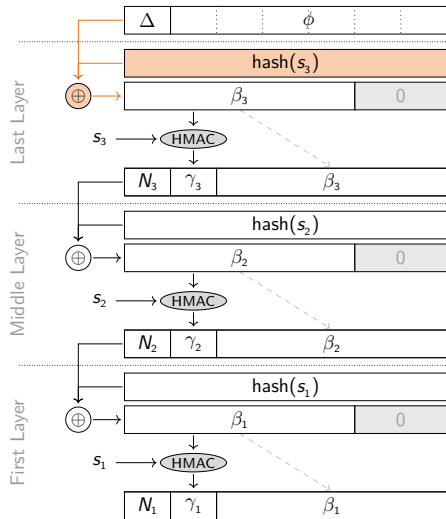
From Sphinx



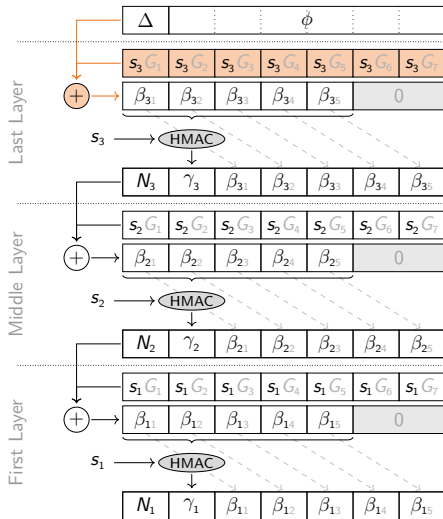
To DSphinx



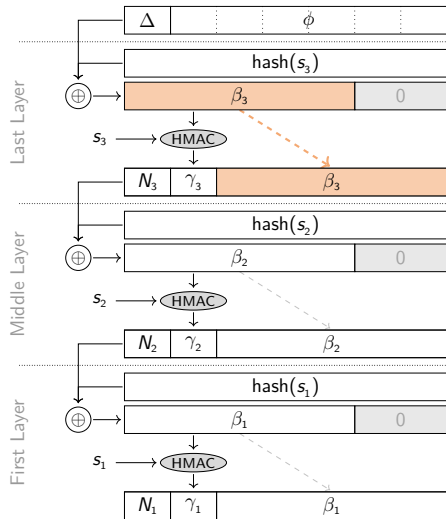
From Sphinx



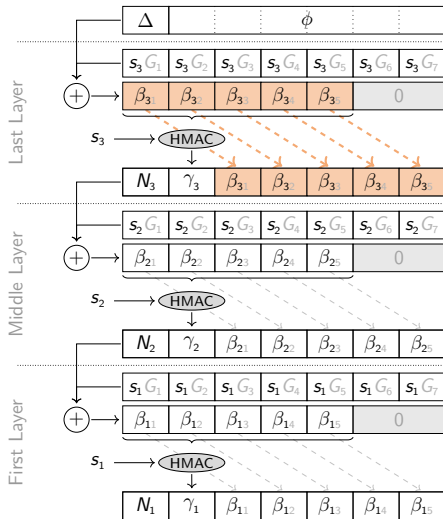
To DSphinx



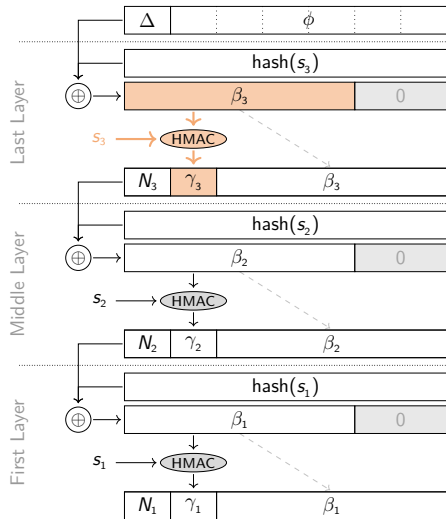
From Sphinx



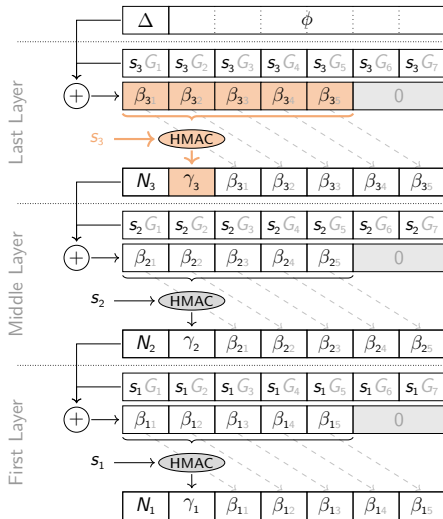
To DSphinx



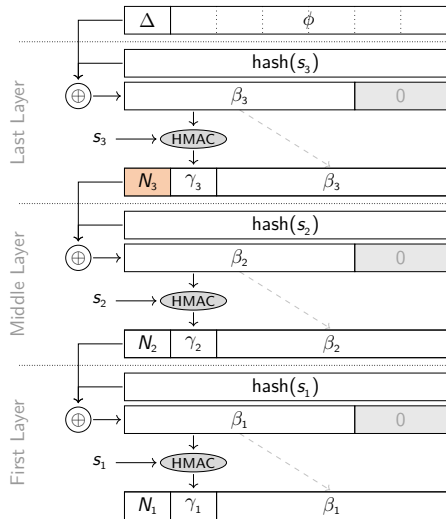
From Sphinx



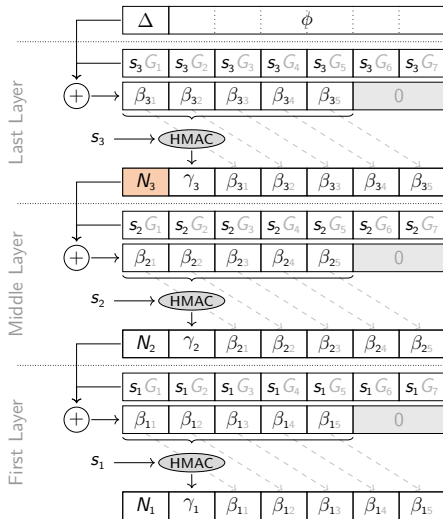
To DSphinx



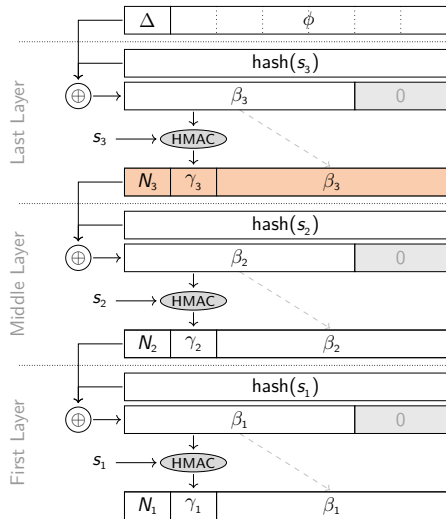
From Sphinx



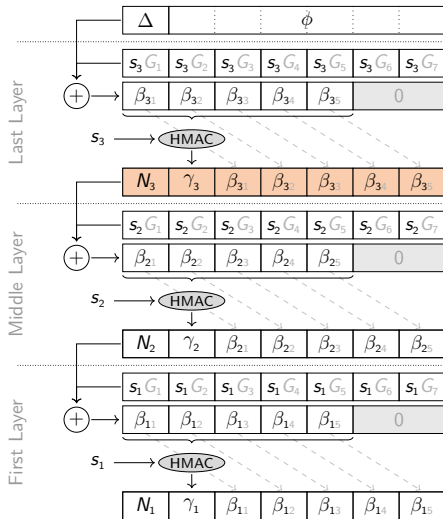
To DSphinx



From Sphinx



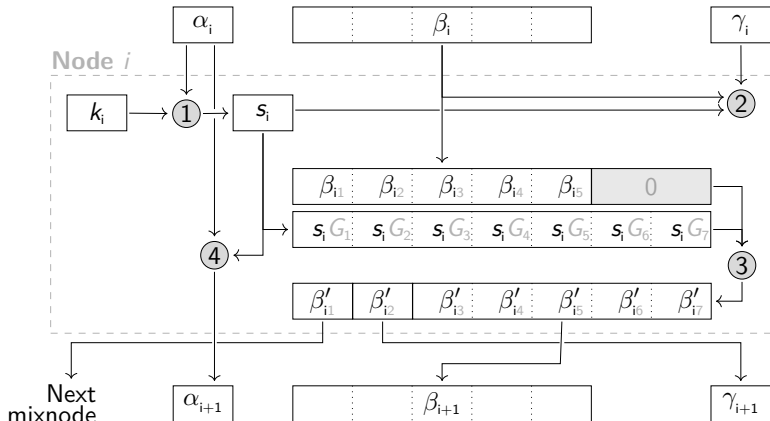
To DSphinx



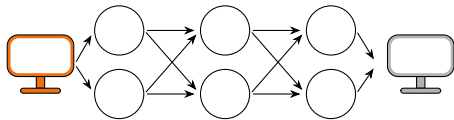
Processing a DSphinx Packet

Steps:

- ① Compute share secret
- ② Integrity verification
- ③ Decrypt header
- ④ Update cryptographics
- ⑤ Decrypt payload

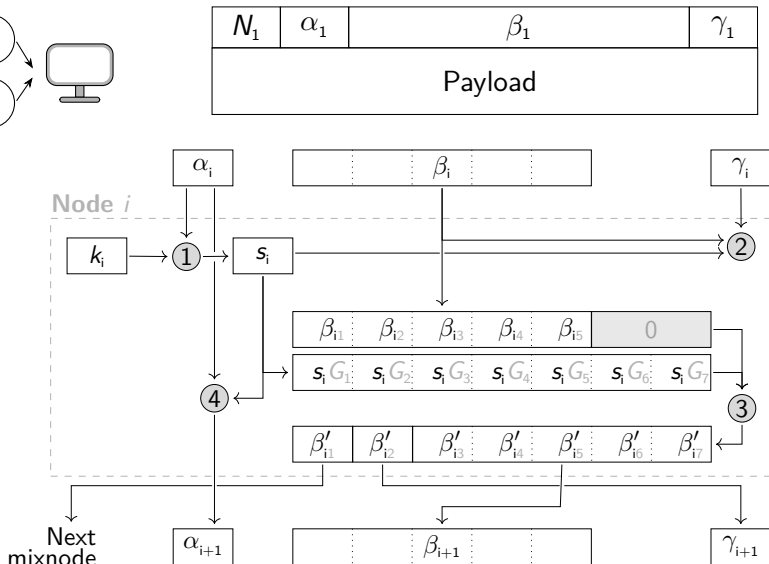


Processing a DSphinx Packet

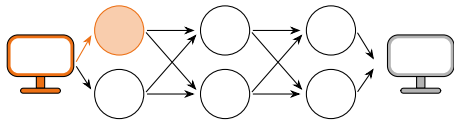


Steps:

- ① Compute share secret
- ② Integrity verification
- ③ Decrypt header
- ④ Update cryptographics
- ⑤ Decrypt payload

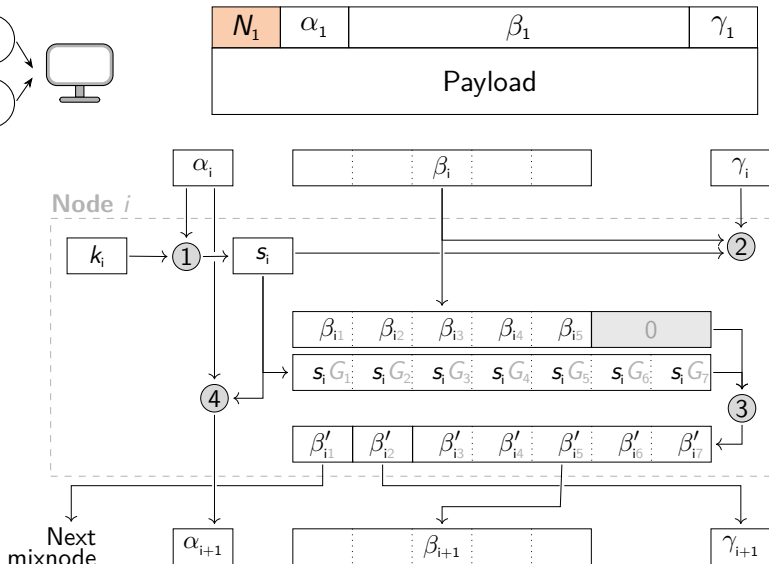


Processing a DSphinx Packet

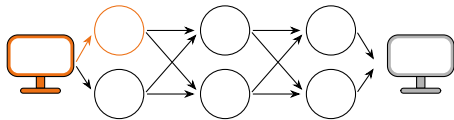


Steps:

- ① Compute share secret
- ② Integrity verification
- ③ Decrypt header
- ④ Update cryptographics
- ⑤ Decrypt payload

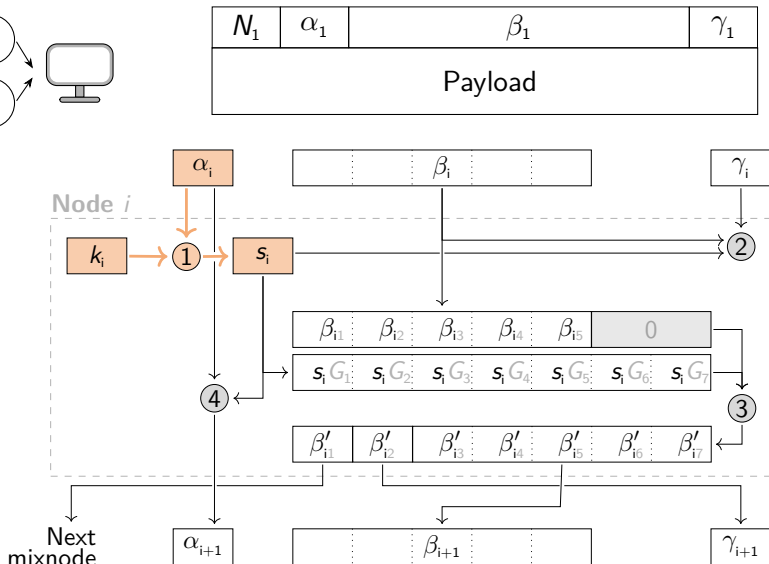


Processing a DSphinx Packet

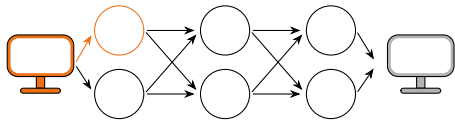


Steps:

- 1 Compute share secret
- 2 Integrity verification
- 3 Decrypt header
- 4 Update cryptographics
- 5 Decrypt payload

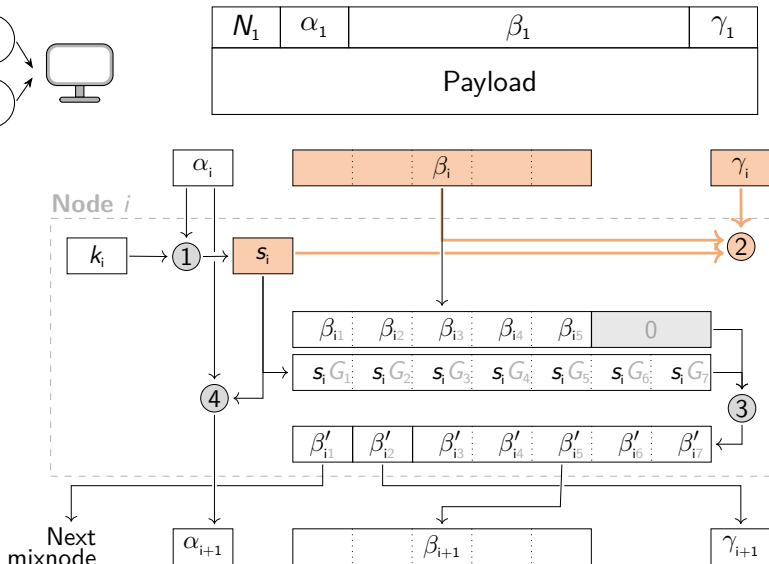


Processing a DSphinx Packet

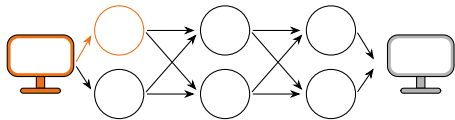


Steps:

- ① Compute share secret
- ② Integrity verification
- ③ Decrypt header
- ④ Update cryptographics
- ⑤ Decrypt payload

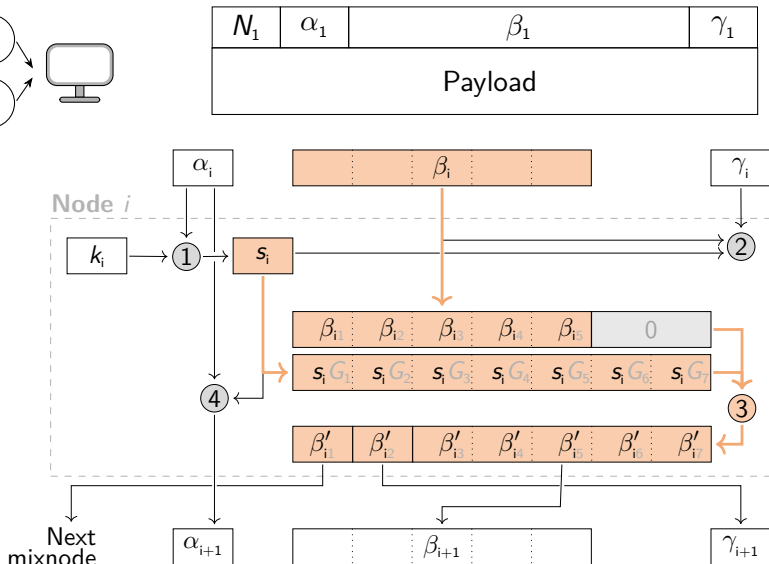


Processing a DSphinx Packet

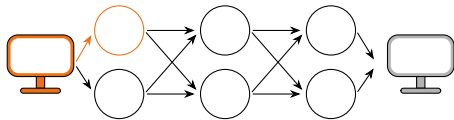


Steps:

- ① Compute share secret
- ② Integrity verification
- ③ Decrypt header
- ④ Update cryptographics
- ⑤ Decrypt payload

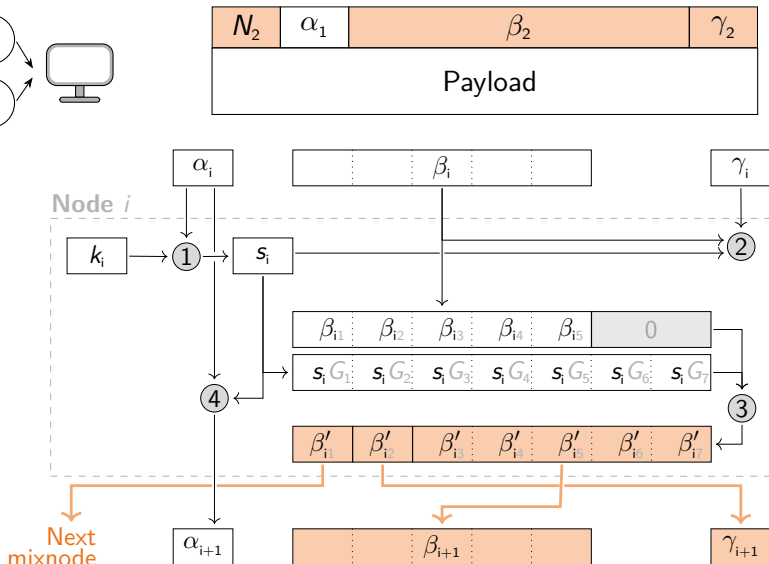


Processing a DSphinx Packet

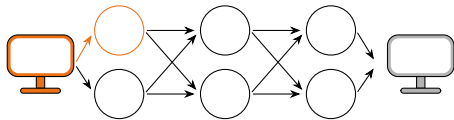


Steps:

- ① Compute share secret
- ② Integrity verification
- ③ Decrypt header
- ④ Update cryptographics
- ⑤ Decrypt payload

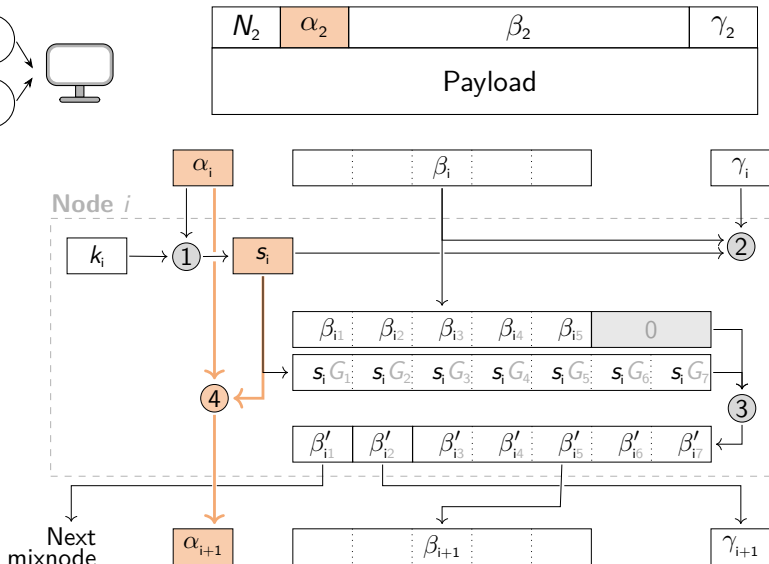


Processing a DSphinx Packet

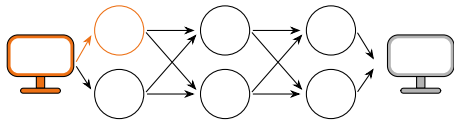


Steps:

- ① Compute share secret
- ② Integrity verification
- ③ Decrypt header
- ④ Update cryptographics
- ⑤ Decrypt payload

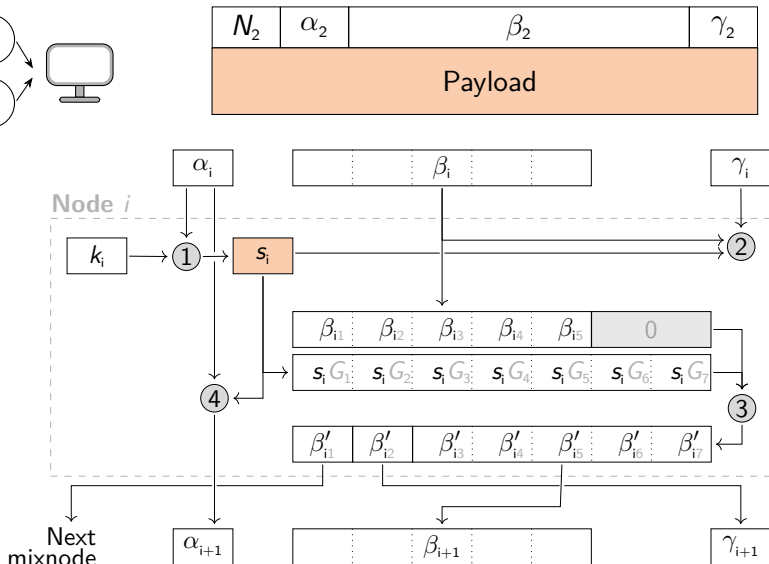


Processing a DSphinx Packet

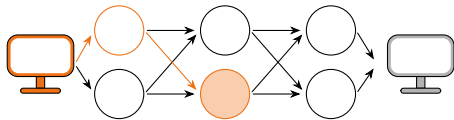


Steps:

- ① Compute share secret
- ② Integrity verification
- ③ Decrypt header
- ④ Update cryptographics
- ⑤ **Decrypt payload**

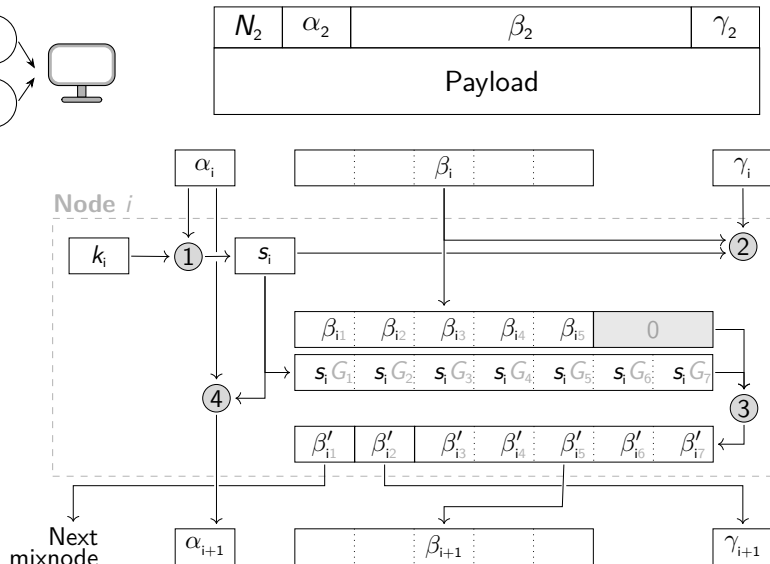


Processing a DSphinx Packet

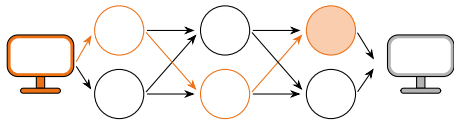


Steps:

- ① Compute share secret
- ② Integrity verification
- ③ Decrypt header
- ④ Update cryptographics
- ⑤ Decrypt payload

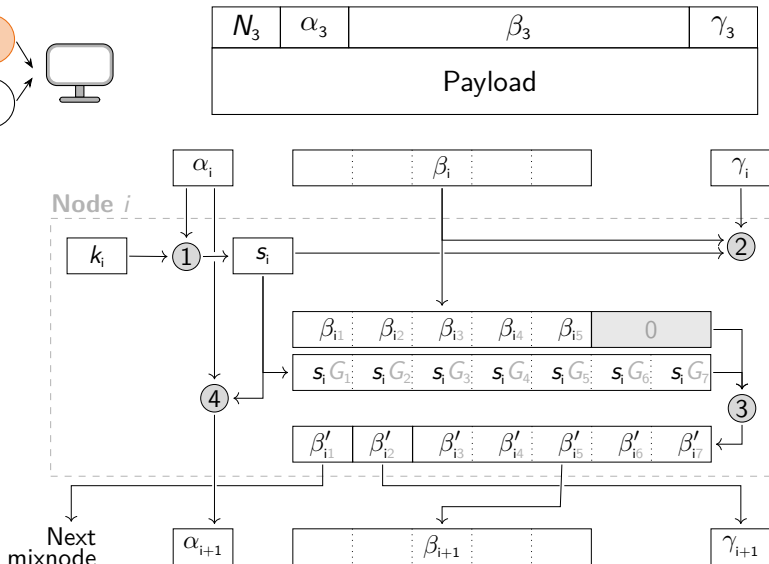


Processing a DSphinx Packet

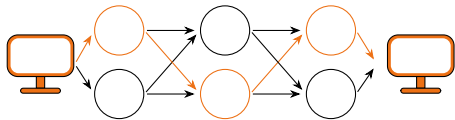


Steps:

- ① Compute share secret
- ② Integrity verification
- ③ Decrypt header
- ④ Update cryptographics
- ⑤ Decrypt payload

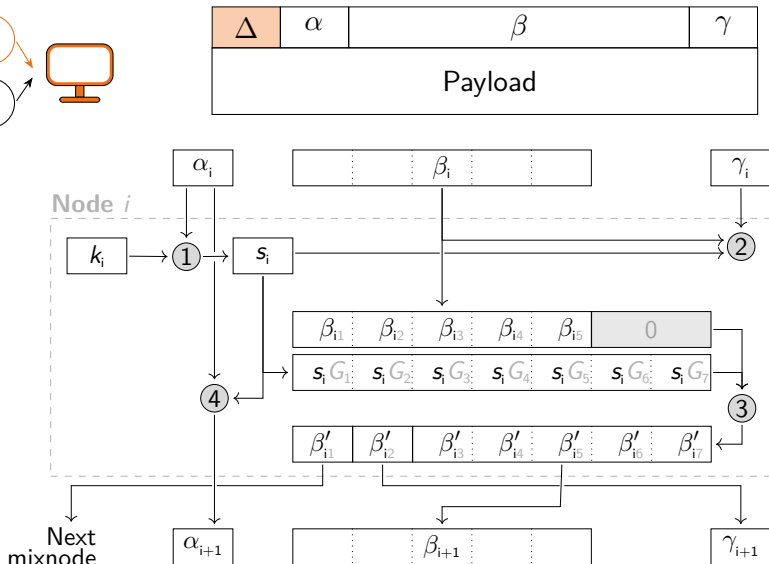


Processing a DSphinx Packet



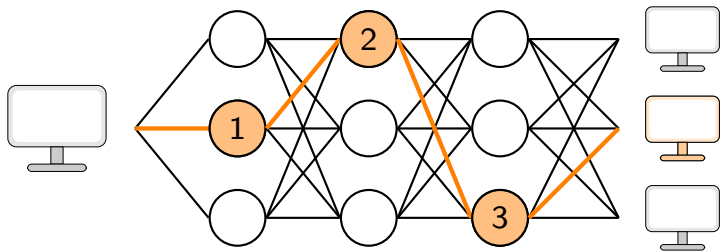
Steps:

- ① Compute share secret
- ② Integrity verification
- ③ Decrypt header
- ④ Update cryptographics
- ⑤ Decrypt payload

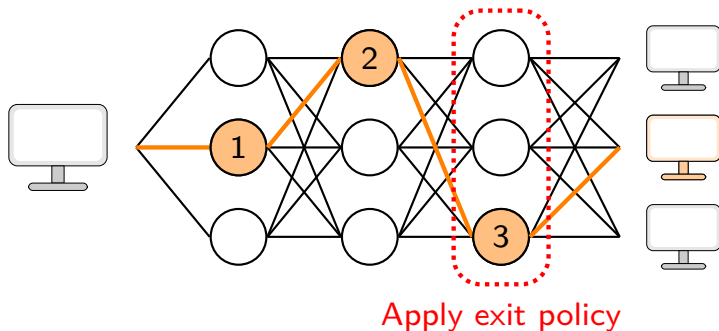


My Research - Credentials

Motivation

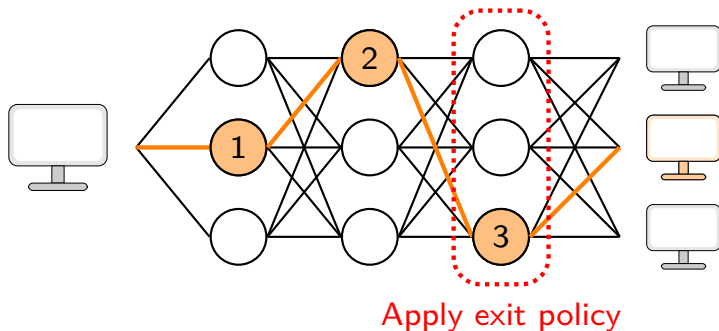


Motivation



Problem: Legitimacy of destination is only verifiable at the **exit node**

Motivation



Problem: Legitimacy of destination is only verifiable at the **exit node**

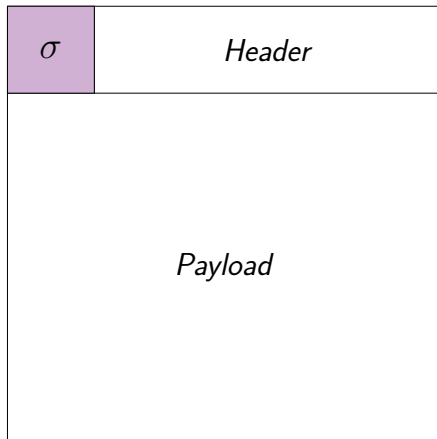
Objective: Legitimacy of destination verifiable at **any hop** without compromising privacy

Objective

Idea: Add a **credential** σ to the packet's header

Desired properties:

- Linked to the final destination
- Verifiable at any hop
- Preserve unlinkability
i.e. updated at each hop
- Preserve client anonymity
i.e. cannot be linked to the client
- Preserve client privacy
i.e. do not leak client information



General Idea

In general, credentials are verified using bilinear pairing such as:

$$e(\sigma, G) == e(M, PK)$$

Credential ← σ M → Message (EC point) PK → Public Key

General Idea

In general, credentials are verified using bilinear pairing such as:

$$e(\sigma, G) == e(H, PK)$$

Credential $\leftarrow$ σ H Header chunks containing destination Δ PK Public Key

Header:



General Idea

Problem: When packet is processed, the final destination chunk Δ is shifted.

Header at node 1:



Header at node 2:



Header at node 3:



Solution: Using all chunks that could contain final destination Δ

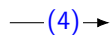
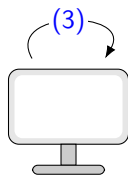
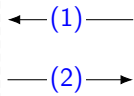
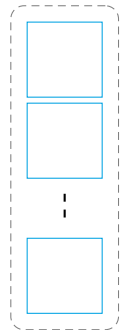
$$e(\sigma, G) == e(N'_2 + N'_3 + \Delta', PK)$$

Header:

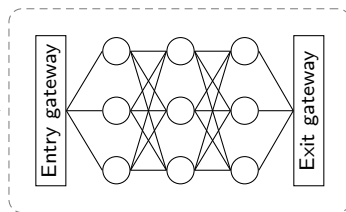


Multi-authorities System Architecture

Authorities



Decentralized Network



- (1) Request
- (2) Issue
- (3) Aggregate
- (4) Rerandomize credential

Assumption: Multi-authorities setup

Credential Construction:

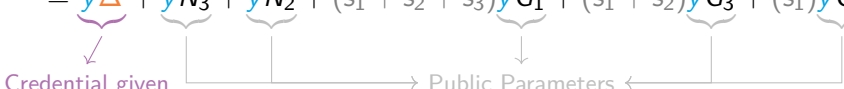
s_i = shared secret *client* - *mixnode_i*

$$\sigma_1 = y (\Delta + N_3 + N_2 + (s_1 + s_2 + s_3)G_1 + (s_1 + s_2)G_3 + (s_1)G_5)$$

Credential Construction & Update

Credential Construction:

s_i = shared secret *client* - *mixnode_i*

$$\begin{aligned}\sigma_1 &= y (\Delta + N_3 + N_2 + (s_1 + s_2 + s_3)G_1 + (s_1 + s_2)G_3 + (s_1)G_5) \\ &= \underbrace{y\Delta}_{\substack{\text{Credential given} \\ \text{by authority}}} + \underbrace{yN_3}_{\text{Public Parameters}} + \underbrace{yN_2}_{\text{Public Parameters}} + (s_1 + s_2 + s_3)\underbrace{yG_1}_{\text{Public Parameters}} + (s_1 + s_2)\underbrace{yG_3}_{\text{Public Parameters}} + (s_1)\underbrace{yG_5}_{\text{Public Parameters}}\end{aligned}$$


Credential Construction & Update

Credential Construction:

s_i = shared secret *client* - *mixnode_i*

$$\begin{aligned}\sigma_1 &= y (\Delta + N_3 + N_2 + (s_1 + s_2 + s_3)G_1 + (s_1 + s_2)G_3 + (s_1)G_5) \\ &= \underbrace{y\Delta}_{\substack{\text{Credential given} \\ \text{by authority}}} + \underbrace{yN_3 + yN_2}_{\text{Public Parameters}} + \underbrace{(s_1 + s_2 + s_3)yG_1 + (s_1 + s_2)yG_3 + (s_1)yG_5}_{\text{Public Parameters}}\end{aligned}$$

Credential Update at each hop:

$$\sigma_{i+1} = \sigma_i - s_i(yG_1 + yG_3 + yG_5 + yG_7) - yN_{i+1}$$

Implementation & Early Results

Implementation

- **Language:** Python 3.14
- **Library:** mclbn256
- **Architecture:** Multi-process
- **Communication:** UDP sockets

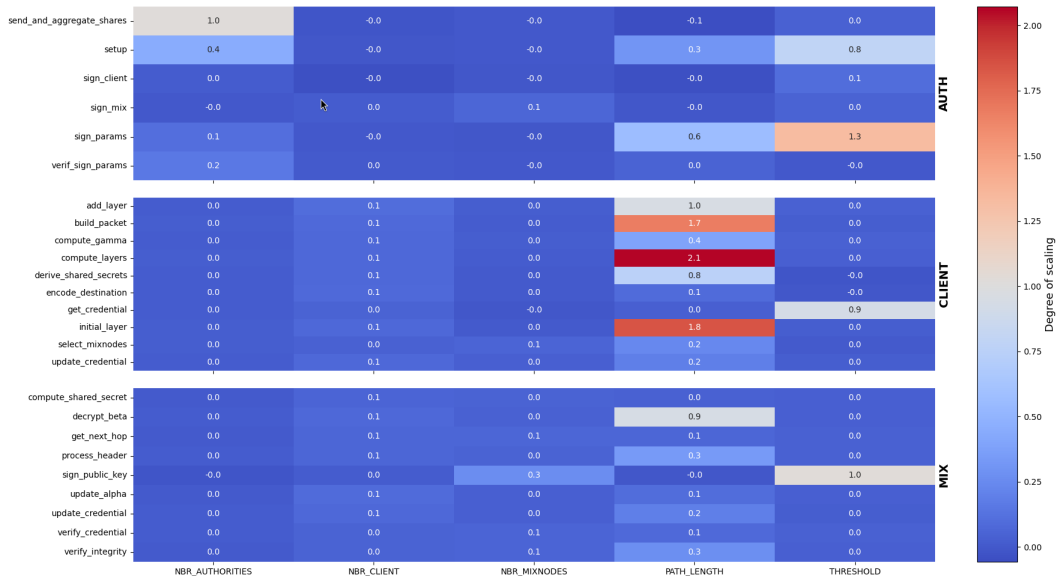
```
[16:37:34.360], CLIENT, 1, <--, AUTH, 9, G1, fc3a0d74, SIGN-CLIENT, , , 1
[16:37:34.361], CLIENT, 1, <--, AUTH, 16, G1, 88cbef0f, SIGN-CLIENT, , , 1
[16:37:34.372], CLIENT, 1, , , , get_credential, CPU: 18.260216 ms, wall:
[16:37:34.373], CLIENT, 1, , , , encode_destination, CPU: 0.619651 ms, wal
[16:37:34.373], CLIENT, 1, , , , select_mixnodes, CPU: 0.050984 ms, wall:
[16:37:34.375], CLIENT, 1, , , , derive_shared_secrets, CPU: 1.974942 ms,
[16:37:34.377], CLIENT, 1, , , , update_credential, CPU: 1.167153 ms, wall
[16:37:34.379], CLIENT, 1, , , , compute_gamma, CPU: 0.267620 ms, wall: 0.
[16:37:34.379], CLIENT, 1, , , , initial_layer, CPU: 2.754870 ms, wall: 2.
[16:37:34.380], CLIENT, 1, , , , compute_gamma, CPU: 0.212643 ms, wall: 0.
[16:37:34.380], CLIENT, 1, , , , add_layer, CPU: 0.853612 ms, wall: 0.8573
[16:37:34.381], CLIENT, 1, , , , compute_gamma, CPU: 0.194506 ms, wall: 0.
[16:37:34.381], CLIENT, 1, , , , add_layer, CPU: 0.816702 ms, wall: 0.8188
[16:37:34.382], CLIENT, 1, , , , compute_gamma, CPU: 0.141348 ms, wall: 0.
[16:37:34.382], CLIENT, 1, , , , add_layer, CPU: 0.752025 ms, wall: 0.7582
[16:37:34.383], CLIENT, 1, , , , compute_gamma, CPU: 0.140052 ms, wall: 0.
[16:37:34.383], CLIENT, 1, , , , add_layer, CPU: 0.758154 ms, wall: 0.7609
[16:37:34.383], CLIENT, 1, , , , compute_layers, CPU: 6.122563 ms, wall: 6
[16:37:34.383], CLIENT, 1, , , , build_packet, CPU: 10.466467 ms, wall: 10
[16:37:34.383], CLIENT, 1, -->, MIX, 1, Header, ca8a1445, HEADER, , , 1
[16:37:34.409], CLIENT, 1, <--, MIX, 34, Header, 523e5f3e, HEADER, , , 1
```

Figure: Log files

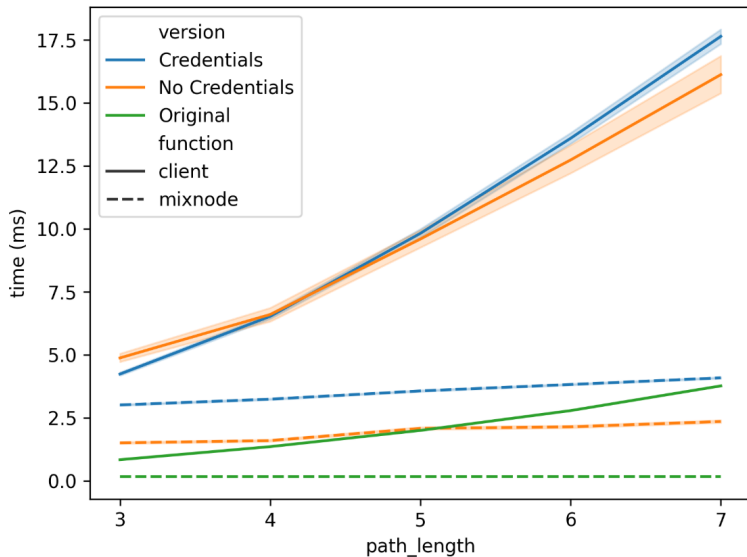
```
AUTH 8 --> CLIENT 1 G1 6d6a909a SIGN-CLIENT,
AUTH 1 --> CLIENT 1 G1 bb44a407 SIGN-CLIENT,
AUTH 7 --> CLIENT 1 G1 6b528491 SIGN-CLIENT,
AUTH 14 --> CLIENT 1 G1 796cb3ad SIGN-CLIENT,
AUTH 18 --> CLIENT 1 G1 3489d2da SIGN-CLIENT,
AUTH 9 --> CLIENT 1 G1 fc3a0d74 SIGN-CLIENT,
AUTH 16 <-- CLIENT 1 G1 b433586d SIGN-CLIENT,
CLIENT 1 <-- AUTH 11 G1 a8e8418c SIGN-CLIENT,
AUTH 16 --> CLIENT 1 G1 88cbef0f SIGN-CLIENT,
CLIENT 1 <-- AUTH 15 G1 c39306c1 SIGN-CLIENT,
CLIENT 1 <-- AUTH 17 G1 61c7ffa2 SIGN-CLIENT,
CLIENT 1 <-- AUTH 7 G1 6b528491 SIGN-CLIENT,
CLIENT 1 <-- AUTH 8 G1 6d6a909a SIGN-CLIENT,
CLIENT 1 <-- AUTH 1 G1 bb44a407 SIGN-CLIENT,
CLIENT 1 <-- AUTH 14 G1 796cb3ad SIGN-CLIENT,
CLIENT 1 <-- AUTH 18 G1 3489d2da SIGN-CLIENT,
CLIENT 1 <-- AUTH 9 G1 fc3a0d74 SIGN-CLIENT,
CLIENT 1 <-- AUTH 16 G1 88cbef0f SIGN-CLIENT,
CLIENT 1 --> MIX 1 Header ca8a1445 HEADER,
MIX 1 <-- CLIENT 1 Header 932c6c88 HEADER,
MIX 1 --> MIX 49 Header a39e2b98 HEADER,
MIX 49 <-- MIX 1 Header ef93a60b HEADER,
MIX 49 --> MIX 24 Header dcdae061 HEADER,
MIX 24 <-- MIX 49 Header 41d8ffec HEADER,
MIX 24 --> MIX 2 Header 7d1a5109 HEADER,
MIX 2 <-- MIX 24 Header 86223ad7 HEADER,
MIX 2 --> MIX 34 Header 718cdd76 HEADER,
MIX 34 <-- MIX 2 Header d5f40b35 HEADER,
MIX 34 --> CLIENT 1 Header 6dc10cbd HEADER,
CLIENT 1 <-- MIX 34 Header 523e5f3e HEADER,
```

Figure: Script running

Results: Scaling degree on CPU time execution for each functions



Results: Latency Comparison with Sphinx Baseline



- Investigate security properties and guarantees
- Finalize the performance benchmarks
- Enhance credential capabilities (e.g. destination blacklisting, validity periods, ...)

- ① DSphinx - A decentralized Sphinx packet construction (using STTPs)
- ② Decentralizing Path sampling for DSphinx (using STTPs)
- ③ Anonymous Credentials for DSphinx (using Multi-Authorities)